claude-windows / 54d4074a-5a8d-4058-b42d-a757c338132c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra\54d4074a-5a8d-4058-b42d-a757c338132c\subagents\workflows\wf_1c9ea2fd-eb8\agent-a6b54849c6782207e.jsonl`
- SHA-256: `8fdeb9ec808bad6c5b6c86d5bf7a80e5e67cf401ddfa528aef264a3cbd5eb87c`
- Source modified: `2026-06-21T19:37:44+00:00`
- Imported at: `2026-07-08T15:59:32+00:00`
- project: `wf_1c9ea2fd-eb8`
- session_id: `54d4074a-5a8d-4058-b42d-a757c338132c`

Transcript

1. user (2026-06-21T19:35:17.948Z)

```
You are the synthesis reviewer. Here are 5 independent structured reviews of the coverage PR as
JSON:

[
  {
    "dimension": "coverage-config",
    "verdict": "ship",
    "findings": [
      {
        "severity": "praise",
        "title": "Excludes are correctly scoped to genuinely non-testable files ? verified by
opening each",
        "detail": "I opened every excluded path and confirmed none hides runtime logic the gate
should cover:\n- src/main.tsx (web/src/main.tsx): pure bootstrap ?
createRoot(...).render(<App/>). The one branch (`if (!root) throw`) is a DOM-precondition guard,
not app logic; excluding the entry is standard and justified.\n- src/api/types.ts
(web/src/api/types.ts): 100% `export interface` declarations, zero emitted runtime code.
Excluding it is correct (and including it would actually create an UNCOVERABLE 0-statement file
? keeping it out avoids a phantom drag).\n- src/test/** ? src/test/setup.ts
(web/src/test/setup.ts): the jsdom localStorage shim. It IS executable code, but it is test-
harness scaffolding that only runs under vitest and ships in no bundle ? correctly excluded.\n-
**/*.d.ts ? src/vite-env.d.ts (web/src/vite-env.d.ts): type-augmentation only (ImportMetaEnv
merge), no runtime. Correct.\nCrucially, files that look config-ish but DO carry logic are NOT
excluded and so still count: src/i18n/index.ts (syncDocumentLang + init) and src/lib/log.ts
(newCorrelationId fallback branch, leveled log()). The exclude list is genuinely minimal.",
        "file": "web/vite.config.ts"
      },
      {
        "severity": "praise",
        "title": "`all: true` + `include: src/**/*.{ts,tsx}` correctly makes untested new files
DROP coverage rather than vanish",
        "detail": "With provider v8, `all: true` instruments every file matched by `include`
even if no test imports it, so a brand-new untested `src/foo.ts` is counted as 0% and pulls the
ratio down ? it cannot be silently ignored. The include glob `src/**/*.{ts,tsx}` matches all
real source (verified against the actual tree: App.tsx, components/*, hooks/*, lib/*,
i18n/index.ts, api/client.ts, etc.), and the comment's claim that deleting a test also surfaces
here is accurate. No competing vitest.config.* exists (confirmed by glob) and package.json adds
no coverage override, so this block is authoritative.",
        "file": "web/vite.config.ts"
      },
      {
        "severity": "praise",
        "title": "All four thresholds sit just below current measured coverage ? green now, and
a real ratchet",
```

```
        "detail": "Floors vs. measured: statements 92 vs 92.61 (slack 0.61), lines 92 vs 92.61
(0.61), branches 87 vs 87.74 (0.74), functions 80 vs 82.35 (2.35). Every floor is BELOW current,
so CI passes; none is at/above current (nothing would fail CI on merge). Vitest treats
thresholds as a `>=` floor, so these are minimums, which is the intended ratchet direction. The
statements/lines/branches floors are tight (sub-1-point slack) and would catch a meaningful
regression ? e.g. one untested ~12-line module added to a ~92% base mathematically pushes
statements below 92.",
        "file": "web/vite.config.ts"
      },
      {
        "severity": "minor",
        "title": "functions floor (80) is the loosest gate ? 2.35 pts of slack lets a couple of
new untested functions slip through unratcheted",
        "detail": "functions is at 82.35 but floored at 80, by far the largest slack of the
four. The inline rationale (App.tsx is an engine-orchestrating shell that's costly to exercise)
is legitimate, but the practical effect is that the functions dimension is the weakest
regression catcher: someone can add new exported helpers and leave them untested as long as the
function ratio stays >= 80, even though the tighter statements/lines gates (0.61 slack) would
usually catch the accompanying lines. This isn't a correctness bug ? it's a tuning gap. Consider
tightening functions to 81 (still 1.35 below current, won't flap) so all four dimensions ratchet
roughly in step, and add a TODO to raise it once App.tsx gets render coverage.",
        "file": "web/vite.config.ts",
        "suggestedFix": "thresholds: { statements: 92, branches: 87, functions: 81, lines: 92 }
// functions: 81 (was 80) ? close the slack while leaving headroom under measured 82.35"
      },
      {
        "severity": "minor",
        "title": "No `perFile` thresholds ? the gate is global, so one well-tested module can
mask a fully untested sibling",
        "detail": "Thresholds are global aggregates (default `perFile: false`). Because the
suite is already ~92%, a newly added file can be 0% covered and the global ratio still clears
the floor if it's small relative to the covered mass ? i.e. a small new module can ride in
untested. `all: true` mitigates this (the 0% file does drag the global number), but the drag is
diluted by repo size. For a FIRST floor this is an acceptable trade-off, but flagging it as the
main residual gaming vector: a 5-10 line untested helper may not move the global ratio below 92.
If you want the gate to bite per-file later, add `perFile: true` (it will require raising
coverage on the currently-weakest files first, so not a drop-in for this PR).",
        "file": "web/vite.config.ts"
      },
      {
        "severity": "nit",
        "title": "Gate is otherwise hard to game trivially ? the obvious dodges are closed",
        "detail": "Checked the common gaming vectors: (1) excluding source to dodge coverage ?
excludes are whitelisted to 4 non-logic paths, can't quietly add app files without it showing in
this diff; (2) tests that import-but-don't-assert to inflate ? v8 counts executed lines not
assertions, so this is a generic limitation of line coverage, not specific to this config; (3)
silently dropping a test ? `all: true` re-counts the now-unexercised source as uncovered,
surfacing the drop. The only non-trivial residual vectors are the functions slack and the
absence of perFile (both filed above). For a first floor, the config is sound; main remaining
risk is that the floors must be RATCHETED UP as coverage grows or they decay into being
meaningless ? the comment acknowledges this but nothing enforces it.",
        "file": "web/vite.config.ts"
      }
    ]
  },
  {
    "dimension": "soundness-and-flakiness",
    "verdict": "ship-with-fixes",
    "findings": [
      {
        "severity": "praise",
        "title": "Determinism / cross-file language leak is correctly handled",
        "detail": "The i18n singleton's global language is reset to 'en' by an awaited
changeLanguage in every relevant hook. Block 1 and Block 3 await changeLanguage('en') in
beforeEach; the kn/hi block (lines 56-81) awaits changeLanguage('en') in afterEach, so kn/hi
```

```
state cannot leak into other test files run in the same vitest process. changeLanguage is always
awaited before render (no async race), and because language is set BEFORE render rather than
mid-render, no act() wrapper is needed ? matching the house pattern in a8.status.test.tsx.
localStorage.clear() in beforeEach plus the setup.ts shim keeps the LanguageDetector from
picking up a persisted pref. This is order-independent and sound.",
        "file": "web/src/components/SelectionFooter.test.tsx"
      },
      {
        "severity": "major",
        "title": "kn/hi 'no ?????? / ????' render assertions are tautological ? the term is not
in any footer key",
        "detail": "In SelectionFooter.test.tsx the kn/hi tests (lines 62-80) render only the
floor-warning block and assert warnText() does not contain the jargon transliteration (?????? /
????). But that term appears in exactly ONE catalog key ? ingest.uriAriaLabel ('engine host') ?
which SelectionFooter never renders. None of footer.floorOne / floorAll / floorSomeLead /
floorSomeRest contains the term in any locale (verified against locales/{kn,hi}/common.json). So
this assertion is always-true regardless of the floor copy: a regression that re-introduced
'engine' jargon into a DIFFERENT footer key, or anywhere the footer doesn't render, would still
pass. The test's own docblock claims it makes 'the bug can't reach the screen' ? but the screen
here can never show that token by construction. The real guard for this lives in catalog-
invariants.test.ts (the engine/server/bucket JARGON test), which is genuinely value-bearing. The
render-level negative adds confidence theater, not coverage.",
        "file": "web/src/components/SelectionFooter.test.tsx",
        "suggestedFix": "Either (a) make the kn/hi tests POSITIVE ? assert the floor warning
renders the expected localized substring (e.g. floorSomeLead's '2 ?????????' / '2 ???????', and
a stable word from floorSomeRest), which actually exercises the kn/hi catalog and ICU plural
rendering; or (b) drop the render-level jargon negatives and rely on catalog-invariants.test.ts
for the token/jargon contract, keeping the render test focused on what only rendering can prove
(Trans component substitution, plural selection, button disabled state)."
      },
      {
        "severity": "minor",
        "title": "min_shot_count render-level negatives are also tautological (token exists in
no catalog)",
        "detail": "warnText().not.toMatch(/min_shot_count/) at SelectionFooter.test.tsx:50,69,79
can never fail: 'min_shot_count' appears in zero catalog strings across all six locales
(verified). The interpolation uses {min} and the component passes values={{ min }}, so the
literal token has no path to the DOM. Same applies to catalog-invariants.test.ts:35-41 ? that
data test is the appropriate home for this invariant (and there it loops ALL_LOCALES, which is
good), but the render-level copies are redundant always-true assertions. Note this is a guard
against a *future* regression, which has some defensive value, but as written it pins a state
that the code cannot currently produce, so it gives false reassurance about the render path
specifically.",
        "file": "web/src/components/SelectionFooter.test.tsx",
        "suggestedFix": "Keep the min_shot_count invariant in catalog-invariants.test.ts
(already correct, loops every locale). Remove or downgrade the render-level
not.toMatch(/min_shot_count/) lines, or convert them into a positive assertion that the {min}
value (6) is actually substituted ? which they already do separately via toContain('6')."
      },
      {
        "severity": "minor",
        "title": "en 'not.toMatch(/engine/i)' negatives are tautological but self-aware",
        "detail": "Lines 33,41,51 assert the en floor copy contains no 'engine'. Verified: no en
footer.* value contains engine/server/bucket. The inline comment ('audit: en is engine-free, so
kn/hi/en stay engine-free') acknowledges this, framing it as a guard. It is harmless but always-
true today; its only value is catching a future en re-wording that adds the word 'engine', which
is a weak and somewhat arbitrary thing to pin. Lower priority than the kn/hi case because at
least 'engine' is an English word that COULD plausibly be typed into en copy.",
        "file": "web/src/components/SelectionFooter.test.tsx"
      },
      {
        "severity": "praise",
        "title": "getByRole('button') single-element assumption is correct in every rendered
state",
        "detail": "SelectionFooter renders exactly one <button> (the Build/Building button); the
```

reel-name field is <input type=\"text\">, which has role textbox, not button. So buildBtn() = getByRole('button') resolves to a single element in the warning, no-warning, building, and empty-selection states. The en plural/summary pins are also correct against the en catalog: footer.count with count=4 ? '4 rallies' (matches /4 rallies/ at line 92), footer.building = 'Building?' (matches /Building/ at line 98), floorOne contains 'skipped' and the {min}=6 substitution, floorAll substitutes {count}=3 and {min}=6, floorSomeLead with atRisk.length=2 ? '2 rallies' (toContain('2')). All verified against locales/en/common.json.",
        "file": "web/src/components/SelectionFooter.test.tsx"
      },
      {
        "severity": "minor",
        "title": "floorSomeLead toContain('2') is a weak numeric pin that can collide with {min}",
        "detail": "At line 47-49 the partial-risk test asserts t.toContain('2') for floorSomeLead count (atRisk.length=2) and t.toContain('6') for {min}. Because warnText() is the WHOLE warning block (lead + rest concatenated), toContain('2') would still pass even if the lead count were rendered wrong, as long as a '2' appeared anywhere (it doesn't today, but the assertion is loose). More importantly it does not prove the plural branch (one vs other) was selected correctly ? '2 rallies' vs a hypothetical '2 rally' both contain '2'. This is brittle-in-reverse: too loose to catch a plural-rule regression. Consider asserting the rendered phrase (e.g. /2 rallies/) so the en plural 'other' branch is actually exercised.",
        "file": "web/src/components/SelectionFooter.test.tsx",
        "suggestedFix": "Replace expect(t).toContain('2') with expect(t).toMatch(/2 rallies/) (en) so the ICU plural 'other' arm is verified, not just the presence of the digit."
      },
      {
        "severity": "nit",
        "title": "Exact-string mistranslation pins in catalog-invariants use not.toBe ? reasonably robust, one is fragile to re-wording",
        "detail": "catalog-invariants.test.ts:96-104 pin specific BAD translations via not.toBe (kn bulk.invert ? '????????', kn/hi result.dismiss ? the harsh forms). not.toBe is the right operator (it only fails if the bad string returns verbatim, so a legitimate re-wording to any OTHER good value still passes). This is the non-brittle way to pin a known-bad value. The only residual fragility: these guard against a single specific regression string each; if the audit's preferred word later changes AND someone independently reintroduces a different bad word, the test won't catch it ? but that's an acceptable scope limit for a regression pin, not a flaw.",
        "file": "web/src/i18n/catalog-invariants.test.ts"
      },
      {
        "severity": "minor",
        "title": "catalog-invariants brand-translit and ingest-keep tests assume keys exist; an absent key passes silently or throws unclearly",
        "detail": "catalog-invariants.test.ts:53-58 asserts e['ingest.intro'] etc. contain 'KhelSutra'. If a future refactor renames/removes ingest.intro, e[key] is undefined and toContain throws a TypeError rather than a clear 'key missing' failure ? acceptable (still red) but noisy. More notably, the brand-translit absence test (47-52) and the jargon test (80-91) are correctly NON-tautological: I verified ??????/???? actually DO appear in the catalog (on ingest.uriAriaLabel), so the jargon test exercises a real allow-list branch (en[key] uses 'engine' ? permitted). Good. Flagging only the undefined-key ergonomics, not a soundness bug.",
        "file": "web/src/i18n/catalog-invariants.test.ts",
        "suggestedFix": "Optionally assert key presence first (expect(e).toHaveProperty(key)) so a renamed key yields a readable failure rather than a TypeError from toContain on undefined."
      },
      {
        "severity": "praise",
        "title": "catalog-invariants is hermetic and order-independent (no i18n singleton touched)",
        "detail": "This file reads locales/*/common.json directly from disk via node:fs and never imports the i18n singleton or changes its language, so it cannot leak kn/hi global state into sibling test files and has no async/act concerns. It runs in the default node environment (no jsdom docblock) consistent with vitest.config.ts. The ALL_LOCALES discovery via readdirSync means newly added locales are automatically covered by the token/code-leak test ? a genuinely robust, non-tautological data invariant.",
        "file": "web/src/i18n/catalog-invariants.test.ts"
      }

```
    ]
  },
  {
    "dimension": "Regression-net coverage gaps in the #77 translation-audit guards (catalog-invariants.test.ts + SelectionFooter.test.tsx)",
    "verdict": "needs-work",
    "findings": [
      {
        "severity": "blocker",
        "title": "Class (d) Hindi over-Sanskritization has NO content guard ? ???????????/?????????/??????? can return verbatim",
        "file": "C:/Users/avidu/Projects/khelsutra-cov/web/src/i18n/catalog-invariants.test.ts",
        "detail": "The #77 audit de-Sanskritized hi on app.badge, app.intro, grid.shots, grid.shotsTitle, grid.shotsConfidence, query.support, result.dismiss. The ONLY assertion protecting any of these is `hi result.dismiss !== '????? ????'`. The stiff terms the audit actually removed ? ??????????? (experimental), ????????? (under development), ??????? (confidence) ? have zero guard. I empirically reverted grid.shots ? '~{count} ????? · ???????????', grid.shotsTitle ? '??? ????? ??????????? ?? (?????????)?', grid.shotsConfidence ? ' ? ???????: {conf}' and ran the full suite: all 15 catalog-invariants tests + 37 hygiene/footer tests PASS. The exact original bug sails straight back in. These keys also never render through SelectionFooter, so its render-guard offers no backstop either.",
        "suggestedFix": "Add a banned-term assertion keyed to the audited fixes, e.g.: const HI_BANNED = ['???????????','?????????','???????']; for each, scan all hi entries and assert none contains it (or scope to the specific keys grid.shots*/query.support). Mirror the self-maintaining style: assert the cleaned replacements ('????????','??? ?? ???','????? ?????') are present on those keys so a re-Sanskritization fails."
      },
      {
        "severity": "major",
        "title": "Class (a) brand: positive 'KhelSutra' check covers only 3 ingest keys, leaving ingest.hint + errors.networkError + errors.invalidUri free to drop the brand",
        "file": "C:/Users/avidu/Projects/khelsutra-cov/web/src/i18n/catalog-invariants.test.ts",
        "detail": "en uses the Latin brand 'KhelSutra' on 6 SPA keys: ingest.intro, ingest.reassure, ingest.fileHint, ingest.hint, errors.networkError, errors.invalidUri (#77 fixed all six ? see the diff: errors.networkError went ?????KhelSutra, errors.invalidUri and ingest.hint went engine?KhelSutra). The guard's positive check (`expect(e[key]).toContain('KhelSutra')`) only iterates ['ingest.intro','ingest.reassure','ingest.fileHint']. So on ingest.hint / errors.networkError / errors.invalidUri an adversary can delete 'KhelSutra' and re-substitute a jargon noun (e.g. revert errors.networkError to '???? ?? ???? ??????') and it passes ? the only thing catching the jargon there would be the class-(c) jargon scan, which (next finding) is also bypassable.",
        "suggestedFix": "Derive the positive-brand key list from en instead of hardcoding 3 keys: for every en entry whose value includes 'KhelSutra', assert the kn/hi value for that key also includes 'KhelSutra'. This is self-maintaining and closes ingest.hint + both errors.* keys."
      },
      {
        "severity": "major",
        "title": "Class (a) brand: the no-transliteration check is an exact-substring match on ONE canonical spelling ? alternate Devanagari/Kannada transliterations slip through",
        "file": "C:/Users/avidu/Projects/khelsutra-cov/web/src/i18n/catalog-invariants.test.ts",
        "detail": "The negative guard filters on the single literal strings ???????? (hi) / ?????????? (kn, which embeds a U+200C ZWNJ). Any other valid transliteration of the brand evades it: I verified '??? ?????' (spaced), '???-?????' (hyphenated), '??????????' (explicit virama+ZWNJ conjunct), '????????' (short-u), and the Kannada '????????' (no ZWNJ) all return False against the guard substring. I put '??? ????? ??? ??? ????' into ingest.hint (an unguarded key) and the suite stayed green ? the brand was transliterated AND the test passed. The task's premise that 'the absence-of-transliteration check should cover that' does NOT hold: it is a denylist of one spelling per locale, not a structural check.",
        "suggestedFix": "Replace the single-string denylist with a positive structural rule: on every key where en contains 'KhelSutra', require the localized value to contain the Latin 'KhelSutra' literal (covers all spellings of the bug at once). Optionally add a coarse Indic-script-run heuristic flagging any Devanagari/Kannada token adjacent to where the brand should be. The positive-presence rule from the previous finding is the robust fix."
      },
```

```
      {
        "severity": "major",
        "title": "Class (c) jargon scan is a fixed 3-term transliteration denylist ? re-leak via
a synonym or alternate spelling is undetected",
        "file": "C:/Users/avidu/Projects/khelsutra-cov/web/src/i18n/catalog-invariants.test.ts",
        "detail": "The self-maintaining engine/server/bucket check is strong for its terms but
only knows the exact strings ??????/????, ??????/?????, ?????/????. The leaked-jargon CONCEPT
can return under a different word: e.g. hi errors.networkError ? '?????? ?? ???? ??????'
(backend) or '????', or kn reassure ? use '?????????'/'???????' (alt for server). It also misses
spelling variants of the same word (????? vs ?????? without final virama; English
'server'/'bucket' written in Latin inside an Indic string). None are in the JARGON array, so all
pass. The SelectionFooter render-guard only asserts absence of ??????/????/min_shot_count on the
floor* keys, so footer leaks via a synonym also slip.",
        "suggestedFix": "Broaden the rule from 'these 3 transliterations' to 'localized body
copy must not introduce an English/Latin tech token where en doesn't': (1) add the Latin words
engine|server|bucket|backend to the denylist scanned in non-en values on keys where en omits
them; (2) extend the Indic term list with known synonyms (?????????, ??????, ???????/???????).
At minimum document that the check is term-bound, not concept-bound."
      },
      {
        "severity": "minor",
        "title": "Class (e)/(d) word-choice guards use exact full-string .not.toBe() ? a
trivially padded/wrapped variant of the banned value passes",
        "file": "C:/Users/avidu/Projects/khelsutra-cov/web/src/i18n/catalog-invariants.test.ts",
        "detail": "The flagged-mistranslation tests assert `load('kn')['bulk.invert'] !==
'????????'`, `kn result.dismiss !== '?????????'`, `hi result.dismiss !== '????? ????'`. Because
these are whole-string equality, the bad term reappears if it is padded or wrapped: '???????? '
(trailing space), '????????.', or '????? ????????' all !== the pinned string and pass while
still shipping the 'rotate' mistranslation to the button. Same for the legalistic ????????? /
harsh ????? ???? ? embedding the word in a longer phrase defeats the pin.",
        "suggestedFix": "Switch from equality to substring containment for the banned token:
`expect(load('kn')['bulk.invert']).not.toContain('????????')`, and likewise
`.not.toContain('?????????')` / `.not.toContain('?????')`. Containment catches padded and
wrapped variants."
      },
      {
        "severity": "minor",
        "title": "Class (b) token leak is well-guarded for the exact token, but a re-leak as a
non-underscore phrasing or a different code tag is uncovered",
        "file": "C:/Users/avidu/Projects/khelsutra-cov/web/src/i18n/catalog-invariants.test.ts",
        "detail": "The min_shot_count / <code> guard (regex /min_shot_count/ and /<\\/?code>/i,
every locale, every key) is the strongest of the five and the SelectionFooter render-test
double-covers footer.floorSomeRest. The residual gap is conceptual leak that isn't that literal
token: the implementation detail can resurface as 'min shot count' / 'minShotCount' / '???????
??? ????', or wrapped in a different inline tag the audit didn't enumerate (e.g. <kbd>, <tt>,
<samp>, or the catalog's own <x> jargon marker reused for an internal token). en's floorSomeRest
is plain prose ('they're too short'), so any inline tag at all in localized floorSomeRest is
suspicious, but only <code> is checked.",
        "suggestedFix": "Generalize the tag check to any inline code-ish tag on body-copy keys:
extend the regex to /<\\/?(code|kbd|samp|tt|var)\\b/i, and add a guard that footer.floorSomeRest
in every locale carries no inline HTML tag that en doesn't (en uses none). Optionally denylist
the spaced/camelCase forms 'min shot count'/'minShotCount'."
      },
      {
        "severity": "praise",
        "title": "The jargon and token guards are genuinely self-maintaining where they apply",
        "file": "C:/Users/avidu/Projects/khelsutra-cov/web/src/i18n/catalog-invariants.test.ts",
        "detail": "Tying the jargon allowance to 'en's value for the same key also uses the
English word' (so uriAriaLabel's 'engine host'/'bucket URL' is permitted without an exception
list) and rendering the REAL catalog through SelectionFooter so a data leak also fails at the
screen are both good designs. The gaps above are about denylist breadth (specific strings vs
concepts) and missing positive assertions for class (d), not about the architecture."
      }
    ]
  },
```

```
  {
    "dimension": "CI wiring + coverage dependency change (web/ SPA coverage floor)",
    "verdict": "ship",
    "findings": [
      {
        "severity": "praise",
        "title": "Q1 ? web job runs test:coverage and the script maps to `vitest run
--coverage`",
        "detail": "C:/Users/avidu/Projects/khelsutra-cov/.github/workflows/test.yml line 48 now
runs `npm run test:coverage` (replacing the old `npm run test`).
C:/Users/avidu/Projects/khelsutra-cov/web/package.json line 13 defines `\"test:coverage\":
\"vitest run --coverage\"`. The original `\"test\": \"vitest run\"` script is retained for local
use. The `--coverage` flag activates the v8 provider + thresholds configured in
C:/Users/avidu/Projects/khelsutra-cov/web/vite.config.ts (statements 92, branches 87, functions
80, lines 92), so a merge that drops below the floor fails CI. Wiring is correct and self-
consistent.",
        "file": "C:/Users/avidu/Projects/khelsutra-cov/.github/workflows/test.yml"
      },
      {
        "severity": "praise",
        "title": "Q2 ? coverage provider is in BOTH package.json devDeps and the lockfile; `npm
ci` resolves clean",
        "detail": "`@vitest/coverage-v8` `^3.2.6` is present in web/package.json devDependencies
(line 33). It also appears 3x in web/package-lock.json: the root manifest mirror
(packages.\"\".devDependencies, line 28), the resolved package node
(node_modules/@vitest/coverage-v8, line 2002, version 3.2.6 with integrity hash). The lockfile
diff is purely additive (751 insertions, 0 deletions) and lockfileVersion 3. I ran `npm ci
--dry-run` in web/ on Node/npm 11.13.0: it resolved 'up to date in 1s' with NO ERESOLVE and no
missing-module error. A clean CI runner will install the provider. No risk of the missing-
lockfile-entry failure the task flagged.",
        "file": "C:/Users/avidu/Projects/khelsutra-cov/web/package-lock.json"
      },
      {
        "severity": "praise",
        "title": "Q3 ? Node 24 is compatible with vitest 3.2.6 / @vitest/coverage-v8 3.2.6",
        "detail": "The lockfile resolves both vitest and @vitest/coverage-v8 to exactly 3.2.6.
The coverage provider's strict peerDependency `\"vitest\": \"3.2.6\"` (lock lines 2026-2028) is
satisfied exactly ? no peer mismatch. vitest 3.2.6 declares engines `node: \"^18.0.0 || ^20.0.0
|| >=22.0.0\"` (lock), and CI's `node-version: \"24\"` satisfies the `>=22.0.0` clause.
@vitest/coverage-v8 has no engines field (inherits none / no extra constraint). No known Node 24
incompatibility for this version pair.",
        "file": "C:/Users/avidu/Projects/khelsutra-cov/web/package-lock.json"
      },
      {
        "severity": "praise",
        "title": "Q4 ? site job is untouched; no duplicate or leftover steps in either job",
        "detail": "The uncommitted diff touches ONLY the web job: a comment update (lines 31-34)
and the single `npm run test` -> `npm run test:coverage` swap (line 48). The site job (lines
8-29: checkout, setup-node@24, npm ci, npm test, shell parse-check) is byte-for-byte unchanged.
The web job's step list is linear with no duplication: checkout -> setup-node -> npm ci ->
typecheck -> test:coverage -> build. No leftover `npm run test` step remains in the web job (it
was replaced, not appended).",
        "file": "C:/Users/avidu/Projects/khelsutra-cov/.github/workflows/test.yml"
      },
      {
        "severity": "nit",
        "title": "Changes are uncommitted working-tree edits, not yet committed/branched",
        "detail": "On branch `chore/web-coverage-floor`, all three files
(.github/workflows/test.yml, web/package.json, web/package-lock.json) show as `M` (modified,
uncommitted) ? `git diff origin/master..HEAD` for these paths is empty because nothing is
committed yet. The change content is correct and ready; this is purely a process note that the
work still needs to be committed (with explicit `git add <paths>` per the repo's concurrent-
checkout safety convention) before it can be reviewed as a PR or land in CI.",
        "file": "C:/Users/avidu/Projects/khelsutra-cov/web/package.json",
        "suggestedFix": "Stage the three files explicitly (git add .github/workflows/test.yml
```

```
web/package.json web/package-lock.json) and commit on the feature branch cut from
origin/master."
      }
    ]
  },
  {
    "dimension": "completeness",
    "verdict": "ship-with-fixes",
    "findings": [
      {
        "severity": "major",
        "title": "QueryBar render test is the single most valuable missing addition (funcs 40%,
the cheapest real win left)",
        "detail": "QueryBar.tsx sits at functions 40% / lines 86.84% (uncovered: lines 37-43
applyText+submit, 68-70/82 pill rendering). The only existing QueryBar test
(web/src/components/QueryBar.examples.test.ts) imports EXAMPLES and exercises the lib/query
parser ? it NEVER renders <QueryBar/>, so the component's own handlers (example-chip tap, form
submit) and the parsed-pill/unreadable branches are entirely untested. This is the exact gap
SelectionFooter just closed, and it is the tightest global metric: functions is 82.35% vs the 80
floor (only +2.35 slack), so lifting QueryBar's 3 uncovered functions is where the coverage
ratchet most needs help. A ~30-line render test mirroring SelectionFooter.test.tsx (render,
fireEvent.change on the input, click an example chip, submit the form, assert onApply fires with
the right source) would move funcs and lines meaningfully and is in-scope for a PR whose stated
goal is 'coverage improved and not regressed.'",
        "file": "web/src/components/QueryBar.tsx",
        "suggestedFix": "Add web/src/components/QueryBar.test.tsx: render <QueryBar
onApply={vi.fn()}/>, fireEvent.change the input to a recognized query and assert the apply
button enables + submit calls onApply(parsed,'typed'); click an example chip and assert
onApply(...,'example'); type an unreadable string and assert the query.unreadable hint renders.
Covers applyText, submit, and the pill branches."
      },
      {
        "severity": "major",
        "title": "Headline '90.44?92.61' conflates denominator reframing with real coverage ?
only +1.25 is attributable to new tests",
        "detail": "Verified locally: re-running coverage with the SAME new exclude config but
with both new test files removed yields 91.36% lines/stmts; with them it is 92.61%. So the
genuine new-test gain is +1.25 pts and it comes ENTIRELY from SelectionFooter.test.tsx
(SelectionFooter.tsx 66%?100%, branches 71?100%, funcs 50?100%). The remaining ~0.9 of the
'90.44?92.61' headline is the denominator reframe from excluding main.tsx (18 lines) and
api/types.ts (91 type-only lines). Those excludes are DEFENSIBLE (v8 otherwise counts type-only
declarations as uncovered runtime lines, and main.tsx is pure bootstrap), so this is not gaming
? but the PR narrative should not imply ~2.2 pts of 'added meaningful coverage' when the real,
test-attributable figure is ~1.25 from one component. Be explicit about the split so the floor
isn't mistaken for a bigger ratchet than it is.",
        "file": "web/vite.config.ts",
        "suggestedFix": "In the PR description (and optionally the vite.config comment), state
the split: excludes account for ~91.36 baseline; the SelectionFooter test adds the real ~1.25 to
92.61. Avoid framing the full 90.44?92.61 as new test coverage."
      },
      {
        "severity": "minor",
        "title": "catalog-invariants.test.ts adds ZERO source-line coverage ? it does not
support the 'coverage improved' claim (but is valuable as a #77 lock)",
        "detail": "catalog-invariants.test.ts reads locales/**/common.json directly via node:fs
(readFileSync/readdirSync) and imports no src/ module, so it contributes nothing to the measured
coverage percentage ? confirmed by the before/after tables being identical except for
SelectionFooter. That is fine and even correct for a data-regression guard, but the PR bundles
it under a 'coverage improved' banner where it carries no coverage weight. Its real value is
locking the #77 audit rules (token/jargon/brand/mistranslation), which is legitimate and worth
keeping ? just don't credit it toward the coverage delta.",
        "file": "web/src/i18n/catalog-invariants.test.ts",
        "suggestedFix": "Frame catalog-invariants in the PR as a content-regression guard
(complements hygiene.test.ts), separate from the coverage-ratchet story; only SelectionFooter
moves the number."
```

```
      },
      {
        "severity": "minor",
        "title": "i18n/index.ts syncDocumentLang fallback stays at branch 25% ? a fair follow-
up, not this PR",
        "detail": "src/i18n/index.ts is branch 25% (uncovered lines 72-76), the
(i18n.resolvedLanguage || lng || DEFAULT_LOCALE).split('-')[0] fallback chain in
syncDocumentLang. This PR does not touch it (25% in both before/after runs). It is genuinely
lower-value than QueryBar ? the extra || arms only fire in degenerate states (no
resolvedLanguage / no lng) and are awkward to drive without stubbing i18n internals ? so a
follow-up is the right call rather than blocking this PR. Worth noting only so it is a conscious
deferral, not an oversight.",
        "file": "web/src/i18n/index.ts",
        "suggestedFix": "Follow-up: a small unit test calling the languageChanged handler /
syncDocumentLang with resolvedLanguage undefined to exercise the lng and DEFAULT_LOCALE fallback
arms; or accept the 25% as a known low-value gap and note it."
      },
      {
        "severity": "nit",
        "title": "Functions floor (80) is the loosest ratchet ? +2.35 slack lets a function-
coverage regression slip silently",
        "detail": "Thresholds: lines/stmts 92 vs actual 92.61 (+0.61), branches 87 vs 87.74
(+0.74), functions 80 vs 82.35 (+2.35). The vite.config comment honestly acknowledges functions
is 'kept looser' because App.tsx is the engine-orchestrating shell. That is reasonable, but
functions is precisely the metric most likely to regress unnoticed given the 2.35-pt gap ? and
App.tsx already sits at funcs 25%. Tightening functions to 82 (in lockstep if the QueryBar
render test lands and lifts it) would make the ratchet bite on the metric it currently protects
least.",
        "file": "web/vite.config.ts",
        "suggestedFix": "After adding the QueryBar render test, raise the functions threshold
toward the new actual (e.g. 82) so the loosest metric gets a real floor; keep the App.tsx-cost
rationale in the comment."
      },
      {
        "severity": "praise",
        "title": "SelectionFooter test is a genuinely strong addition ? coverage AND behavior
regression in one",
        "detail": "SelectionFooter.test.tsx renders the REAL en/kn/hi catalogs and asserts the
cleaned #77 copy (no min_shot_count token, no engine/server jargon, correct Build-disabled logic
across allDropped/partial/empty/building states). It simultaneously drives SelectionFooter.tsx
66?100% (the previously-uncovered floor-warning block) and guards the #77 fixes at the render
layer, so a re-introduced token/jargon fails on-screen, not just in the data test. This is
exactly the right kind of coverage ? exercising real branches with meaningful assertions rather
than smoke-rendering for line credit.",
        "file": "web/src/components/SelectionFooter.test.tsx"
      }
    ]
  }
]

Produce a single consolidated verdict. Deduplicate overlapping findings. Classify each UNIQUE
finding by severity. Decide an overall verdict: 'ship' (merge as-is), 'ship-with-fixes' (small
fixes first), or 'needs-work'. For any blocker/major, give the exact file + concrete fix. Be
decisive and concise ? the goal is to know what (if anything) MUST change before merge versus
what is optional/follow-up.
```

2. assistant (2026-06-21T19:35:21.976Z)

I'll synthesize these 5 reviews. Let me first verify the most consequential claim ? the `needs-work` blocker about Hindi over-Sanskritization having no guard ? since it's the only thing that could block merge and it directly contradicts the "ship" votes elsewhere.

3. user (2026-06-21T19:35:22.683Z)

No files found

4. user (2026-06-21T19:35:22.810Z)

```
No files found
```

5. assistant (2026-06-21T19:35:26.256Z)

```
The files aren't in the default cwd. The CI review referenced a worktree path (`khelsutra-cov`).
Let me locate the actual files.
```

6. user (2026-06-21T19:35:27.953Z)

```
/c/Users/avidu/Projects/khelsutra-cov
---
/c/Users/avidu/Projects/khelsutra-cov/web/src/components/SelectionFooter.test.tsx
/c/Users/avidu/Projects/khelsutra-cov/web/src/i18n/catalog-invariants.test.ts
```

7. user (2026-06-21T19:35:30.685Z)

```
1       // Content regression guards for the localization catalogs ? the rules the adversarial
translation
2       // audit (#77) established, pinned so the specific bugs can never silently return. This
complements
3       // hygiene.test.ts (which proves key/placeholder PARITY) by asserting catalog CONTENT:
4       //   1. no internal code-token / <code> leaks in any locale,
5       //   2. the brand "KhelSutra" is never transliterated into an Indic script,
6       //   3. engine/server/bucket jargon appears ONLY on keys where the en source uses it,
7       //   4. the specific flagged mistranslations stay fixed.
8       // These are pure data assertions over repo-root /locales (D-CATALOG-HOME) ? no render
context needed.
9       import { describe, it, expect } from "vitest";
10      import { readFileSync, readdirSync } from "node:fs";
11      import { join } from "node:path";
12
13      const LOCALES_DIR = join(import.meta.dirname, "..", "..", "..", "locales");
14
15      type Flat = Record<string, string>;
16      function flatEntries(obj: Record<string, unknown>, prefix = ""): Flat {
17        const out: Flat = {};
18        for (const [k, v] of Object.entries(obj)) {
19          const key = prefix ? `${prefix}.${k}` : k;
20          if (v && typeof v === "object" && !Array.isArray(v)) Object.assign(out,
flatEntries(v as Record<string, unknown>, key));
21          else out[key] = String(v);
22        }
23        return out;
24      }
25      const load = (loc: string): Flat =>
26        flatEntries(JSON.parse(readFileSync(join(LOCALES_DIR, loc, "common.json"), "utf8")) as
Record<string, unknown>);
27
28      const ALL_LOCALES = readdirSync(LOCALES_DIR, { withFileTypes: true })
29        .filter((d) => d.isDirectory())
30        .map((d) => d.name);
31      const en = load("en");
32
33      describe("catalog content ? no internal token / HTML-tag leaks (every locale)", () => {
34        for (const loc of ALL_LOCALES) {
35          it(`${loc}: no 'min_shot_count' code token or <code> tag in any user-facing string`,
() => {
36            const offenders = Object.entries(load(loc))
37              .filter(([, v]) => /min_shot_count/.test(v) || /<\/?code>/i.test(v))
38              .map(([k]) => k);
39            expect(offenders).toEqual([]);
40          });
```

```
41        }
42      });
43
44      describe("catalog content ? the brand is never transliterated (Indic locales)", () => {
45        const TRANSLIT: Record<string, string> = { kn: "??????????", hi: "????????" };
46        for (const loc of ["kn", "hi"] as const) {
47          it(`${loc}: no Indic transliteration of "KhelSutra" anywhere`, () => {
48            const offenders = Object.entries(load(loc))
49              .filter(([, v]) => v.includes(TRANSLIT[loc]))
50              .map(([k]) => k);
51            expect(offenders).toEqual([]);
52          });
53          it(`${loc}: the ingest console keys keep the Latin brand`, () => {
54            const e = load(loc);
55            for (const key of ["ingest.intro", "ingest.reassure", "ingest.fileHint"]) {
56              expect(e[key], `${loc} ${key}`).toContain("KhelSutra");
57            }
58          });
59        }
60      });
61
62      describe("catalog content ? engine/server/bucket jargon only where en uses it (Indic
locales)", () => {
63        // en deliberately avoids these terms in user-facing body copy (it says "KhelSutra" /
"cloud-storage"
64        // / passive voice) ? EXCEPT the technical uriAriaLabel ("engine host", "bucket URL").
So the rule is
65        // self-maintaining: a localized jargon term is allowed ONLY on a key whose en value
also uses the
66        // English word. Reintroducing "engine"/"server"/"bucket" anywhere else fails here.
67        const JARGON: Record<string, ReadonlyArray<readonly [string, string]>> = {
68          kn: [
69            ["??????", "engine"],
70            ["??????", "server"],
71            ["?????", "bucket"],
72          ],
73          hi: [
74            ["????", "engine"],
75            ["?????", "server"],
76            ["????", "bucket"],
77          ],
78        };
79        for (const loc of ["kn", "hi"] as const) {
80          it(`${loc}: a jargon term appears only on keys whose en value also uses it`, () => {
81            const e = load(loc);
82            const violations: Array<{ key: string; term: string; en: string }> = [];
83            for (const [key, val] of Object.entries(e)) {
84              for (const [term, eng] of JARGON[loc]) {
85                if (val.includes(term) && !(en[key] ?? "").toLowerCase().includes(eng)) {
86                  violations.push({ key, term, en: en[key] ?? "(absent)" });
87                }
88              }
89            }
90            expect(violations).toEqual([]);
91          });
92        }
93      });
94
95      describe("catalog content ? flagged mistranslations stay fixed (Kannada/Hindi word
choice)", () => {
96        it("kn bulk.invert is not the 'rotate' mistranslation (????????)", () => {
97          expect(load("kn")["bulk.invert"]).not.toBe("????????");
98        });
99        it("kn result.dismiss is not the legalistic '?????????'", () => {
100         expect(load("kn")["result.dismiss"]).not.toBe("?????????");
```

```
101       });
102       it("hi result.dismiss is not the harsh '????? ????'", () => {
103         expect(load("hi")["result.dismiss"]).not.toBe("????? ????");
104       });
105     });
106
```

8. assistant (2026-06-21T19:35:35.376Z)

Now let me verify the core blocker claim. The reviewer asserts the Hindi over-Sanskritization terms (???????????, ?????????, ???????) appear in the #77 audit but have no guard, and that reverting them keeps tests green. Let me check what the #77 diff actually changed and whether these terms are truly unguarded. First, let me look at the actual hi catalog and the git history.

9. user (2026-06-21T19:35:36.987Z)

```
9f1ec9c fix(i18n): native-review pass on the kn/hi SPA console catalogs (#77)
9a2d7d2 feat(site): A7 ? camera-placement diagram on the capture page (#76)
aa8d5e9 feat(site): A6 ? plain-language Request-Access GPU/storage questions (#75)
1cd84af feat(site): A3 ? localize recording-education + marketing to kn/hi, Noto on the site
(#74)
c1848df feat(web): LW2 ? send the UI locale to /api/compile (watermark) + /api/process
(analytics) (#73)
1b1fa7b feat(web+site): B3 ? trilingual marketing site via shared t()/data-i18n shim (D-CATALOG-
HOME) (#70)
f7e2bd5 chore(site): add robots.txt + a "crawling / bot protection" runbook for khelsutra.guru
(#71)
bea8a2a docs(appliance): flip T11 retention sweep to ? ? tool shipped (engine #299), scheduling
tracked (#301) (#69)
=== BRANCH ===
chore/web-coverage-floor
=== STATUS ===
 M .github/workflows/test.yml
 M web/package-lock.json
 M web/package.json
 M web/vite.config.ts
?? web/src/components/SelectionFooter.test.tsx
?? web/src/i18n/catalog-invariants.test.ts
```

10. assistant (2026-06-21T19:35:40.436Z)

Note: the worktree status shows `vite.config.ts` as modified and the two test files as untracked (uncommitted) ? consistent with the CI reviewer's process note. Let me verify the #77 diff to see exactly which hi keys/terms were de-Sanskritized.

11. user (2026-06-21T19:35:42.994Z)

```
-    "badge": "?????-???? ??? · ??????",
+    "badge": "???? ??????? ????? · ??????",
-    "intro": "<b>???? ?? ?????</b> ?? ???? ?? ??????? ???? ? ???? <i>????? ???? ??? 15?</i> ?
??? ????? ??? ???? ?????? ?? ???????? ????? ?? ??? <b>?????</b> ???? ???? ??; ?? ?? ???? ?????
?????? ??????"
+    "intro": "<b>???? ?? ?????</b> ?? ???? ?? ??????? ???? ? ???? <i>????? ???? ??? 15?</i> ?
??? ????? ??? ???? ?????? ?? ?????? ?????? ??? ?? ??? <b>?????</b> ???? ??; ?? ?? ???? ?????
?????? ??? ????"
-    "support": "<b>???? ?? ?????</b> ?? ???? ?? ??????? ???? ???? ??? ??????? ??? ???-?????
<x>???????????</x> ?? (?????????)?"
+    "support": "<b>???? ?? ?????</b> ?? ???? ?? ??????? ???? ???? ??? ??? ???? ??? ???-?????
<x>????????</x> ?? (??? ?? ??? ??)?"
-    "shots": "~{count} ????? · ???????????",
-    "shotsTitle": "??? ????? ??????????? ?? (?????????) ?? ??? ?? ???? ??{conf}? ???? ???? ????
?? ????? ?? ??????? ?????",
-    "shotsConfidence": " ? ???????: {conf}"
```

```
+    "shots": "~{count} ????? · ????????",
+    "shotsTitle": "??? ????? ???????? ?? (??? ?? ??? ??) ?? ??? ?? ???? ??{conf}? ???? ????
???? ?? ????? ?? ??????? ?????",
+    "shotsConfidence": " ? ????? ?????: {conf}"
-    "intro": "???????? ?? ?? ?????? ?????? ? ?? ???? ???? ?? ???? ?? ????? ????? ? ?? ??
??????? ????? ????? ????? ??? ???? ?? ???? ????",
+    "intro": "KhelSutra ?? ??? ?????? ?????? ?? ?? ???? ??? ??????? ????? ???? ? ????? ??? ????
?? ???? ????",
```

12. assistant (2026-06-21T19:35:47.925Z)

The #77 diff confirms the de-Sanskritization claim: ??????????????????, ????????????? ?? ??? ??, ????????????? ????? on `query.support`, `grid.shots`, `grid.shotsTitle`, `grid.shotsConfidence`. Now let me empirically verify the blocker ? that reverting these terms keeps the suite green (i.e., they have NO guard). Let me check the `catalog-invariants` test against a reverted catalog.

13. user (2026-06-21T19:35:52.016Z)

```
=== Do the banned hi terms appear ANYWHERE in catalog-invariants.test.ts? ===
0

=== Do they appear in SelectionFooter.test.tsx? ===
0

=== Confirm current hi values (post-#77, cleaned) ===
    "support": "<b>???? ?? ?????</b> ?? ???? ?? ??????? ???? ???? ??? ??? ???? ??? ???-?????
<x>????????</x> ?? (??? ?? ??? ??)?"
    "shots": "~{count} ????? · ????????",
    "shotsTitle": "??? ????? ???????? ?? (??? ?? ??? ??) ?? ??? ?? ???? ??{conf}? ???? ???? ????
?? ????? ?? ??????? ?????",
    "shotsConfidence": " ? ????? ?????: {conf}"
```

14. assistant (2026-06-21T19:35:56.626Z)

Confirmed: the de-Sanskritization terms appear nowhere in either test file ? class (d) Hindi over-Sanskritization is genuinely unguarded. Now let me run the actual empirical test the `needs-work` reviewer claimed: revert one of these keys and confirm the suite stays green. This is the decisive check.

15. user (2026-06-21T19:36:05.517Z)

```
Reverted grid.shots + grid.shotsConfidence to Sanskritized forms
=== Running catalog-invariants + SelectionFooter tests against reverted catalog ===

 RUN  v3.2.6 C:/Users/avidu/Projects/khelsutra-cov/web

 ? src/i18n/catalog-invariants.test.ts (15 tests)
15ms
(node:44508) ExperimentalWarning: localStorage is not available because --localstorage-file was
not provided.
(Use `node --trace-warnings ...` to show where the warning was created)
 ? src/components/SelectionFooter.test.tsx (9 tests)
241ms

 Test Files  2 passed (2)
      Tests  24 passed (24)
   Start at  01:06:02
   Duration  2.58s (transform 111ms, setup 26ms, collect 416ms, tests 256ms,
environment 1.38s, prepare 400ms)
```

16. assistant (2026-06-21T19:36:11.182Z)

The blocker is confirmed empirically: with `grid.shots` reverted to the Sanskritized

`???????????` and `shotsConfidence` to `???????`, all 24 tests still pass green. The exact #77 bug for class (d) sails straight back in. Let me restore the catalog and quickly verify the two `major` denylist-evasion claims (unguarded brand keys + alternate transliteration), then run the full suite to confirm baseline green.

17. user (2026-06-21T19:36:15.470Z)

```
Restored hi catalog
=== Verify class(a) brand: which en keys contain KhelSutra vs which the guard checks ===
en keys WITH KhelSutra: ['ingest.intro', 'ingest.hint', 'ingest.reassure', 'ingest.fileHint',
'errors.networkError', 'errors.invalidUri', 'site.hero.lead', 'site.record.intro',
'site.record.upload.body', 'site.capture.step4.b2']
guard positively checks only: ingest.intro, ingest.reassure, ingest.fileHint
UNGUARDED brand keys: ['ingest.hint', 'errors.networkError', 'errors.invalidUri',
'site.hero.lead', 'site.record.intro', 'site.record.upload.body', 'site.capture.step4.b2']
```

18. assistant (2026-06-21T19:36:22.269Z)

The class (a) brand finding is confirmed: en uses "KhelSutra" on 10 keys, but the positive guard only checks 3 (`ingest.intro/reassure/fileHint`). `ingest.hint`, `errors.networkError`, `errors.invalidUri` (all explicitly fixed in #77 per the diff) are unguarded for brand-drop. Now let me confirm the alternate-transliteration evasion and then verify the full suite is green at baseline (the coverage floor claim).

19. user (2026-06-21T19:36:29.838Z)

```
Injected spaced-translit + machine into ingest.hint
=== Run guards against spaced-transliteration injection ===

 ? src/i18n/catalog-invariants.test.ts (15 tests)
14ms

 Test Files  1 passed (1)
      Tests  15 passed (15)
   Start at  01:06:29
   Duration  550ms (transform 41ms, setup 15ms, collect 28ms, tests 14ms,
environment 0ms, prepare 191ms)

Restored
```

20. assistant (2026-06-21T19:36:35.071Z)

Both `major` denylist-evasion findings confirmed empirically: a spaced transliteration `??? ?????` plus the jargon "machine" injected into the unguarded `ingest.hint` passes all 15 invariant tests. Now the final check ? confirm the actual coverage floor passes (the "ship" claim from coverage-config and CI reviewers) so I know the baseline is genuinely green.

21. user (2026-06-21T19:36:45.656Z)

```
=== catalog restored clean ===

=== Running full coverage suite (the CI gate) ===
  ...ionFooter.tsx |     100 |      100 |     100 |     100 |
  StatusChip.tsx   |     100 |      100 |     100 |     100 |
  TapFilters.tsx   |     100 |      100 |     100 |     100 |
 src/hooks         |   92.89 |    76.92 |     100 |   92.89 |
  useIngestJob.ts  |   92.52 |    78.57 |     100 |   92.52 | ...08-110,141-142
  ...ySelection.ts |     100 |    83.33 |     100 |     100 | 9
  useUpload.ts     |    91.8 |    70.58 |     100 |    91.8 | 76-78,82-83
 src/i18n          |     100 |       25 |     100 |     100 |
  index.ts         |     100 |       25 |     100 |     100 | 72-76
 src/lib           |   97.22 |    94.61 |     100 |   97.22 |
  errorText.ts     |     100 |      100 |     100 |     100 |
  errors.ts        |     100 |      100 |     100 |     100 |
```

```
  floor.ts         |     100 |      100 |     100 |     100 |
  format.ts        |     100 |      100 |     100 |     100 |
  log.ts           |   81.81 |     87.5 |     100 |   81.81 | 13-14
  query.ts         |   97.07 |    93.45 |     100 |   97.07 | ...95,284,286,288
  selection.ts     |     100 |     92.3 |     100 |     100 | 30
-------------------|---------|----------|---------|---------|-------------------

=============================== Coverage summary ===============================
Statements   : 92.61% ( 1266/1367 )
Branches     : 87.74% ( 365/416 )
Functions    : 82.35% ( 70/85 )
Lines        : 92.61% ( 1266/1367 )
================================================================================
```

22. assistant (2026-06-21T19:37:00.494Z)

The full suite passes with measured coverage exactly matching all five reviews: statements 92.61, branches 87.74, functions 82.35, lines 92.61 ? all above the configured floors (92/87/80/92). The coverage gate is genuinely green, QueryBar.tsx confirms the funcs gap (let me note it's there), and the i18n branch 25% matches the completeness review.

I've now independently verified the load-bearing claims across all five reviews:
- **Coverage floor passes** (statements 92.61 / branches 87.74 / functions 82.35 / lines 92.61, all ? floor) ? the "ship" votes are correct.
- **The `needs-work` blocker is real**: class (d) Hindi over-Sanskritization (???????????/???????) has zero guard; reverting `grid.shots` + `grid.shotsConfidence` to the original buggy strings leaves all 24 tests green.
- **Two `major` denylist gaps are real**: brand guard only positively checks 3 of 10 KhelSutra keys (ingest.hint, errors.networkError, errors.invalidUri unguarded), and a spaced transliteration `??? ?????` + jargon passes.

Now I'll synthesize. The key judgment: is the blocker a merge-blocker for *this PR*? The PR's stated goal is a coverage floor + a #77 regression net. The de-Sanskritization class was an explicit part of #77, and the new test file claims to lock #77's rules ? but silently fails to cover class (d). That's a real correctness gap in the deliverable, not a nice-to-have. However, it does not make the coverage gate itself wrong or unsafe to merge. I'll land this as **ship-with-fixes**: the coverage/CI machinery is sound and shippable, but the catalog-invariants guard must gain a class (d) assertion (and the brand-key generalization) before it can be credited as a #77 lock, since otherwise it ships false reassurance.

23. user (2026-06-21T19:37:39.458Z)

Structured output provided successfully

24. assistant (2026-06-21T19:37:44.633Z)

Synthesis complete. Verdict: **ship-with-fixes**.

The coverage/CI machinery is verified-green and shippable (I ran the full suite: 92.61/87.74/82.35/92.61, all ? floor), but the #77 regression-net guard has three real, reproduced gaps that must be closed before it can be honestly credited as a #77 lock ? chiefly the class (d) Hindi over-Sanskritization with zero coverage (reverting the buggy terms leaves all 24 tests green). One self-maintaining fix (derive guards positively from en's own values) closes all three.